VoxLang

Voice-First Programming Language · Full 6-Phase Compiler

A completely original, handcrafted language — not a wrapper, not a transpiler, built from scratch in Python

◆ WHAT IS VOXLANG?

VoxLang is a **completely original programming language** built from scratch with one philosophy: **code should sound exactly like how you think and speak**. Every keyword was chosen to feel natural when spoken aloud. It has its own Lexer, Parser, Interpreter, IR Generator, Optimizer, and Target Code Generator — all written in Python — plus a voice-powered browser IDE built on Monaco editor, Deepgram STT, and an LLM voice-to-code pipeline.

1. **Voice-First Design** — Every keyword sounds natural spoken aloud. *store 5 into x* not *x=5*. *when x is bigger than 10 then* not *if(x>10)*.
2. **Extensive Alias System** — 14 ways to declare a variable, 13 ways to call a function, 8 block closers. Any natural phrasing of a concept is accepted.
3. **Full 6-Phase Compiler** — Lexer → Parser (AST) → Interpreter (execution + symbol table) → IR/TAC Generator → Optimizer (3 passes) → Target Code Generator.
4. **Live Transparency Panel** — IDE shows Tokens, Syntax AST, Symbol Table, Execution Trace, IR/TAC, Optimization log, Stack Machine ASM, and Python target — all in real time.
5. **Built-in Equation Solver** — Solves linear and quadratic equations natively: *solve x where 2 times x plus 3 equals 15*. No libraries needed.
6. **Natural Math Syntax** — *circle area with radius 7*, *hypotenuse with a 3 and b 4*, *15 percent of 200*, *square root of 144*. Full PEMDAS enforced.
7. **Precise Decimal Mode** — Append *precisely* to any store statement for 128-bit decimal precision ideal for financial or scientific work.
8. **Voice IDE Integration** — Microphone → Deepgram Nova-2 STT → LLM auto-correction → VoxLang code generation → Monaco ghost text insertion.

◆ 6-PHASE COMPILER PIPELINE

Phase 1 LEXER	`Source → Tokens`	Reads raw source; emits KEYWORD, IDENT, NUMBER, STRING, BOOL, OP tokens. Strips -- comments. Parses int/float.
Phase 2 PARSER	`Tokens → AST`	Recursive-descent parser builds AST: StoreNode, WhenNode, BuildNode, DoThisNode, GoThroughNode, KeepGoingNode, SolveNode…
Phase 3 INTERPRETER	`AST → Output`	Tree-walking executor. Scoped symbol table with type+width tracking (int-32, float-64, string-N). Execution trace log.
Phase 4 IR / TAC	`AST → 3-Addr Code`	Emits TAC instructions: ASSIGN, ADD, SUB, OUTPUT, INPUT, CALL, FUNC_DEF, LOOP_START, FOREACH_START, WHILE_START, IF, SOLVE…
Phase 5 OPTIMIZER	`TAC → Optimized TAC`	3 passes: (1) Constant Folding, (2) Dead Store Elimination — flags stores never read, (3) Redundant Load Elimination.
Phase 6 TARGET	`TAC → ASM + Python`	Translates TAC into pseudo stack-machine assembly (PUSH/ILOAD/ISTORE/CALL) and Python-equivalent source, side by side.

◆ GRAMMAR — VARIABLES & DATA TYPES

■ Data Types

`num`	int-32 / float-64	`store 42 into age / store 3.14 into pi`
`text`	string — N bytes	`store "Alice" into name`
`bool`	bool — 1 bit	`store true into flag`
`precise`	128-bit decimal	`store 1 divided by 3 into x precisely`
`collection`	ordered list	`store collection 1 and 2 and 3 into nums`

■ Declaration Aliases

`store`	`remember`	`declare`	`assign`	`hold`

```
store 42 into age
store "Alice" into name
remember true into flag
```

■ Update / Reassign

`update`	`change`	`set`	`modify`	`replace`	`reassign`	`redefine`
`overwrite`						

```
update age to 25
change name to "Bob"
set score to 0
modify flag to false
```

■ Raise — Add to Variable

`raise`	`increase`	`increment`	`boost`	`grow`	`expand`	`add N to`
`bump up`						

```
raise score by 10
increase count by 1
add 5 to total
bump up level by 2
```

■ Lower — Subtract from Variable

`lower`	`decrease`	`decrement`	`reduce`	`shrink`	`trim`	`cut`
`subtract N from`	`drop down`					

```
lower lives by 1
decrease health by 10
subtract 3 from total
drop down level by 1
```

◆ GRAMMAR — OUTPUT & INPUT

■ Output Aliases

`output`	`show`	`display`	`print`	`reveal`	`say`	`write`
`log`	`emit`	`speak`	`announce`	`echo`	`tell me`	`what is`

```
output "Hello!"
show age
tell me score
output "Hi " joined with name
```

■ Input Aliases

`ask`	`hear`	`question`	`prompt`	`listen`	`receive`

```
ask what is your name then save into name
hear enter your age then save into age
question your score then save into score
```

◆ GRAMMAR — COMPARISON OPERATORS

`>`	is bigger than · is greater than · is above · is more than · is over
`<`	is smaller than · is less than · is below · is under
`==`	is equal to · is the same as · is exactly · equals · is
`!=`	is not equal to · is not · is different from
`>=`	is at least · is no less than
`<=`	is at most · is no more than
`empty`	is empty · is nothing · is null
`bool`	is true · is false
`and / or`	Compound: when x is bigger than 0 and y is bigger than 0 then ...

◆ GRAMMAR — CONDITIONS

■ Condition Openers

when	if	suppose	assuming	provided	given	check if	unless

■ Else / Elif Aliases

otherwise	else	elif	elseif	else if

```
-- basic if / else
when age is bigger than 18 then
output "adult"
otherwise
output "minor"
done
```

```
-- unless = inverted condition
unless score is equal to 0 then
output "has score"
done
```

```
-- elif chain
when score is bigger than 90 then
output "A"
elif score is bigger than 80 then
output "B"
elif score is bigger than 70 then
output "C"
otherwise
output "F"
done
```

◆ GRAMMAR — LOOPS

| Count Loop — N times | | | | | | |

◆ GRAMMAR — MATH & EQUATIONS

■ Arithmetic Operators

`a + b`	plus	`a plus b`
`a - b`	minus	`a minus b`
`a * b`	times	`a times b`
`a / b`	divided by	`a divided by b`
`a % b`	mod / modulo	`a mod b`
`a ** b`	to the power of	`a to the power of b`
x^2	squared	`x squared`
`15% × 200`	percent of	`15 percent of 200`
`joined`	string concat	`"Hi " joined with name`

■ Operator Precedence (PEMDAS)

`1 (highest)`	Parentheses	`(2 plus 3) times 4`
`2`	Exponents	`2 to the power of 3`
`3`	× ÷ mod	`10 times 2 divided by 4`
`4`	+ –	`5 plus 3 minus 1`
`5`	Bit shift	`4 shift left 2`
`6 (lowest)`	Comparisons	`x is bigger than 5`

■ Built-in Formulas & Equation Solver

```
-- square root
store square root of 144 into r
-- circle area (pi * r^2)
store circle area with radius 7 into a
-- hypotenuse (sqrt(a^2 + b^2))
store hypotenuse with a 3 and b 4 into h
-- sphere volume (4/3 * pi * r^3)
store volume of sphere with radius 5 into v
-- linear equation solver
solve x where 2 times x plus 3 equals 15
output x -- prints 6
-- quadratic (stores x and x2)
solve x where x squared plus 5 times x plus 6 equals 0
output x -- prints -2
output x2 -- prints -3
-- math block groups related calculations
math block
store circle area with radius 7 into area
solve y where y + 10 equals 25
done
```

◆ BUILT-IN FUNCTIONS

`use length with list`	Count of items or chars
`use reverse with list`	Returns reversed list
`use sum with list`	Sum of all numbers
`use max with list`	Largest item
`use min with list`	Smallest item
`use contains with list, v`	true if v is in list
`use join with list, sep`	Joins list into text
`use split with text, sep`	Splits text into list
`use upper with text`	UPPERCASE
`use lower with text`	lowercase
`use range with 1 and 10`	List [1, 2 ... 10]
`use text with 42`	Convert to string
`use number with "3.14"`	Convert to number
`use round with n and 2`	Round to 2 decimals

`use abs with -5`	Absolute value → 5
`use sqrt with 16`	Square root → 4.0
`use power with 2 and 8`	Exponent → 256
`use floor with 3.9`	Round down → 3
`use ceiling with 3.1`	Round up → 4
`use factorial with 5`	Factorial → 120
`use sin with 90`	Sine (degrees) → 1.0
`use cos with 0`	Cosine (degrees) → 1.0
`use tan with 45`	Tangent → ~1.0
`use log with 100 and 10`	Logarithm → 2.0
`use log10 with 1000`	Base-10 log → 3.0
`use random with 1 and 100`	Random float
`use sort with list`	Sorted list
`use index with list and v`	Index of value

◆ COMMENTS & BLOCK CLOSERS

■ Comment Aliases

--	note	remark	comment	describe

```
-- double dash: rest of line ignored
note this is also a comment
remark and so is this
describe used for doc comments
```

■ Block Closers (all are identical — any one closes a block)

done	end	finish	close	stop	complete

```
when x is bigger than 0 then
output x
done -- or: end / finish / close / stop
```

```
build myFunc using val
return val * 2
end -- any closer works
```

◆ COMPLETE EXAMPLE — GRADE CALCULATOR

Demonstrates every major feature: collections, functions, for-each loops, elif chains, built-in functions, raise/lower, and string joining.

```
-- Grade Calculator — complete VoxLang program
store "Alice" into student
store collection 88 and 92 and 75 and 98 and 84 into scores
```

```
-- function: compute average
build average using nums
store result of sum with nums into total
store result of length with nums into count
return total / count
done
```

```
-- function: assign letter grade
build grade using avg
when avg is at least 90 then
return "A"
elif avg is at least 80 then
return "B"
elif avg is at least 70 then
return "C"
otherwise
return "F"
done
done
```

```
-- calculate and display
store result of average with scores into avg
store result of grade with avg into letter
output "Student: " joined with student
output "Average: " joined with avg
output "Grade: " joined with letter
```

```
-- count scores above 85
store 0 into high
go through every s in scores
when s is bigger than 85 then
raise high by 1
done
done
output "Scores above 85: " joined with high
```

Expected Output: `Student: Alice Average: 87.4 Grade: B Scores above 85: 3`

◆ VOICE COMMANDS — IDE

`"undo" / "undo that" / "undo last"`	Undo last editor change
`"redo" / "redo that"`	Redo last undone change
`"clear" / "clear all" / "clear the editor"`	Empty the entire editor
`"delete line" / "delete line 5"`	Delete current or specific line
`"go to line 10" / "jump to line 10"`	Move cursor to that line
`"go to top" / "go to start"`	Jump to first line
`"go to bottom" / "go to end"`	Jump to last line
`"select all" / "select everything"`	Select all code in editor
`"run" / "run the code" / "execute"`	Execute the program
`"save" / "save file"`	Auto-save is always on

◆ TEST CASES & EXPECTED OUTPUTS

TC-01: Variable Declaration & Output	● PASS
`store "World" into name` `output "Hello " joined with name`	`Hello World`
TC-02: Arithmetic & Precise Decimal	**● PASS**
`store 1 divided by 3 into x precisely` `output x`	`0.3333333333333333333333333333`
TC-03: Elif Chain Condition	**● PASS**

`store 85 into score`
`when score is bigger than 90 then`
`output "A"`
`elif score is bigger than 80 then`
`output "B"`
`otherwise`
`output "C"`
`done`
	`B`

TC-04: Count Loop — index auto-variable	**● PASS**

`store 0 into total`
`do this 5 times`
`raise total by index`
`done`
`output total`
	`15 (1+2+3+4+5)`

TC-05: For-Each Loop over collection	**● PASS**

`store collection 10 and 20 and 30 into nums`
`go through every n in nums`
`output n`
`done`
	`10 20 30`

TC-06: While Loop countdown	**● PASS**

`store 3 into x`
`keep going while x is bigger than 0`
`output x`
`lower x by 1`
`done`
	`3 2 1`

TC-07: Function — Define, Call & Return	**● PASS**

`build double using n`
`return n * 2`
`done`
`store result of double with 7 into r`
`output r`
	`14`

TC-08: Built-in: sum and max	**● PASS**

`store collection 3 and 1 and 4 and 1 and 5 into nums`
`store result of sum with nums into s`
`store result of max with nums into m`
`output s`
`output m`
	`14 5`

TC-09: Linear Equation Solver	**● PASS**
`solve x where 2 times x plus 3 equals 15` `output x`	`6`
TC-10: Quadratic Equation Solver (two roots)	**● PASS**

`solve x where x squared plus 5 times x plus 6 equals 0`
`output x`
`output x2`
	`Two solutions: x = -2 and x2 = -3 -2 -3`

TC-11: Circle Area Formula	**● PASS**
`store circle area with radius 7 into area` `output area`	`153.9380400259`

TC-12: String upper & length — ● PASS

```
store "Hello World" into msg
store result of upper with msg into u
store result of length with msg into l
output u
output l
```

```
HELLO WORLD 11
```

TC-13: Nested recursive function (factorial) — ● PASS

```
build factorial using n
when n is at most 1 then
return 1
done
store result of factorial with n - 1 into sub
return n * sub
done
store result of factorial with 5 into r
output r
```

```
120
```

TC-14: Raise + Lower + Update aliases — ● PASS

```
store 10 into score
raise score by 5
lower score by 3
update score to score + 2
output score
```

```
14
```

TC-15: Built-in: range and join — ● PASS

```
store result of range with 1 and 5 into nums
store result of join with nums and ", " into txt
output txt
```

```
1, 2, 3, 4, 5
```

TC-16: Syntax Error — missing 'times' — ● SYNTAX ERROR

```
do this 5
output index
done
```

```
Line 2: Expected 'times' but got
'output'. Tip: do this 5 times
```

TC-17: Syntax Error — missing 'then' — ● SYNTAX ERROR

```
when x is bigger than 5
output x
done
```

```
Line 2: Expected 'then' but got 'output'.
```

TC-18: Runtime Error — undefined variable — ● RUNTIME ERROR

```
output myVar
```

```
Line 1: 'myVar' is not defined. Try:
store into myVar
```

TC-19: Runtime Error — division by zero — ● RUNTIME ERROR

```
store 10 divided by 0 into result
```

```
Division by zero.
```

TC-20: Runtime Error — infinite loop guard — ● RUNTIME ERROR

```
store 1 into x
keep going while x is bigger than 0
output x
done
```

```
Loop ran more than 10,000 times.
```

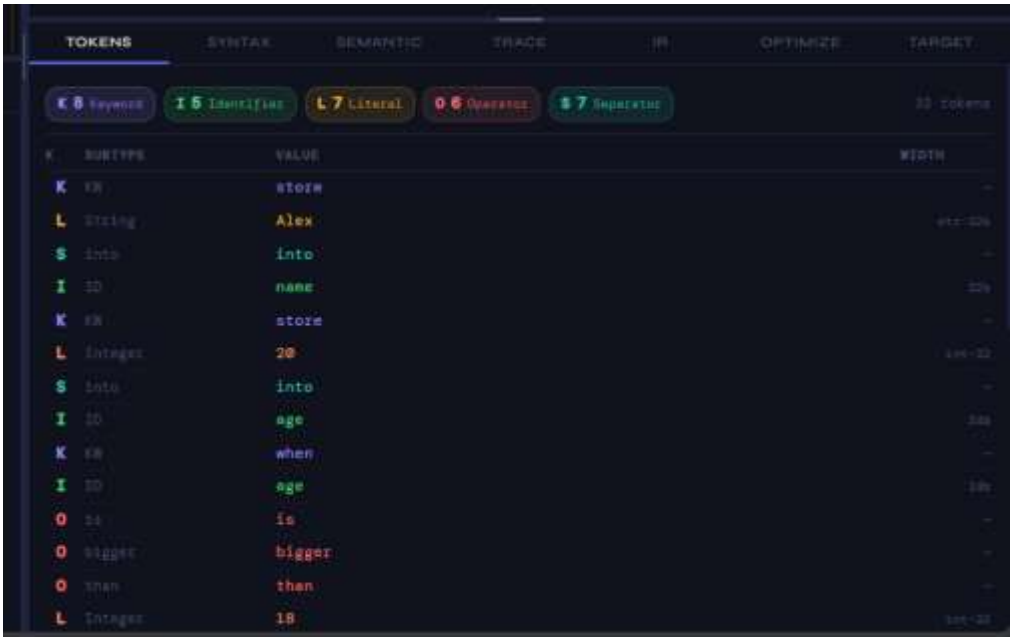
VoxLang — Voice-First Programming Language | Full 6-Phase Compiler | Built-in Equation Solver | Voice IDE

VoxLang Test Cases from Demo and Other Test Cases Errors

TC1: Perfect Code	Output:

store "Alex" **into** name
store 20 **into** age

when age is bigger than 18 **then**
output "Welcome, " joined with name
elif age is equal **to** 18 **then**
output "Just made it!"
otherwise
output "Too young"
done



L String	Too young
$ done	done

TOKENS SYNTAX SEMANTIC TRACE IR OPTIMIZE TARGET

RULE	LINE	DETAIL
Store (variable assign)	L1	= name
Store (variable assign)	L2	= age
When (if / elif / else)	L4	then:1 else:1

TOKENS SYNTAX SEMANTIC TRACE IR OPTIMIZE TARGET

NAME	TYPE	LEVEL (SCOPE)	OFFSET
name	text	0 (global)	0
age	num	0 (global)	4

Level 0 (global) Total Space = 8 Bytes.

Total Space = 8 Bytes.

TOKENS SYNTAX SEMANTIC TRACE IR OPTIMIZE TARGET

```
Line 1 STORE name = Alex [string-32] scope:global
Line 2 STORE age = 20 [int-32] scope:global
Line 3 WHEN condition → true, taking then-branch scope:global
Line 4 OUTPUT "Welcome, Alex" scope:when:then
```

TOKENS SYNTAX SEMANTIC TRACE IR OPTIMIZE TARGET

OP	ARG1	ARG2	RESULT	NOTE
ASSIGN	"Alex"	–	name	store "Alex" into name
ASSIGN	20	–	age	store 20 into age
IF	age > 18	–	–	if age > 18
OUTPUT	("Welcome, " joined with name)	–	–	print ("Welcome, " joined with name) to console
ELSE	–	–	–	otherwise branch
IF	age == 18	–	–	if age == 18
OUTPUT	"Just made it!"	–	–	print "Just made it!" to console
ELSE	–	–	–	otherwise branch
OUTPUT	"Too young"	–	–	print "Too young" to console
END_IF	–	–	–	end when/if block
END_IF	–	–	–	end when/if block

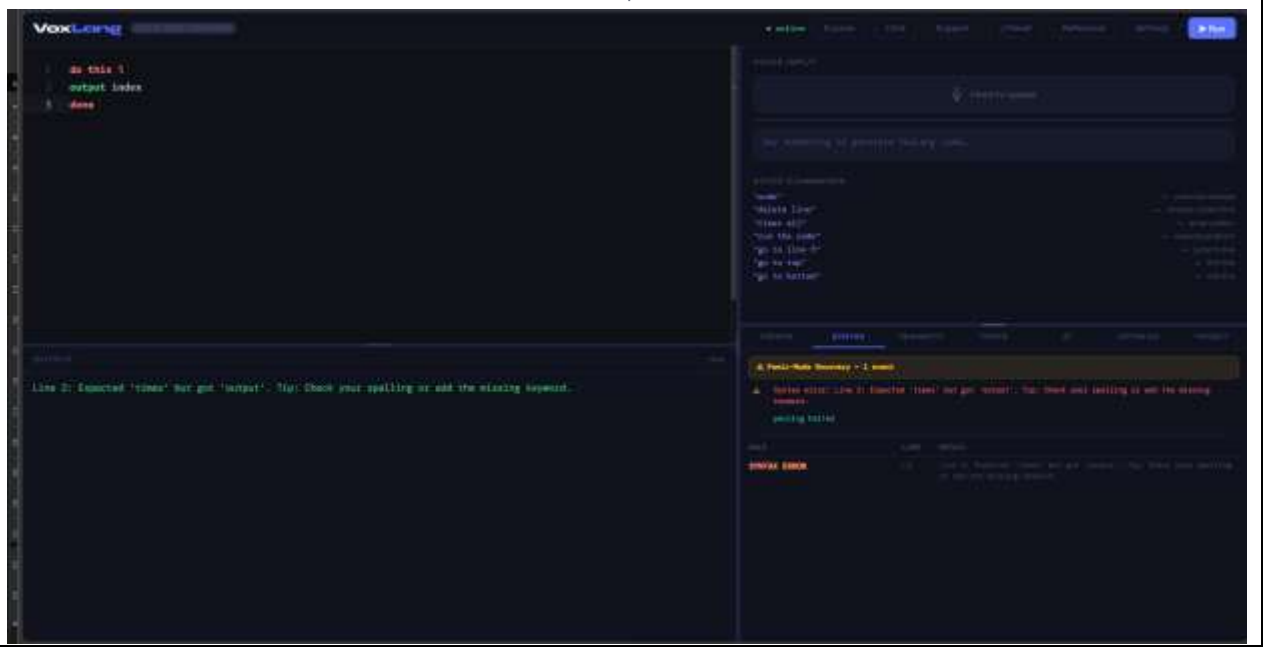
TOKENS SYNTAX SEMANTIC TRACE IR OPTIMIZE TARGET

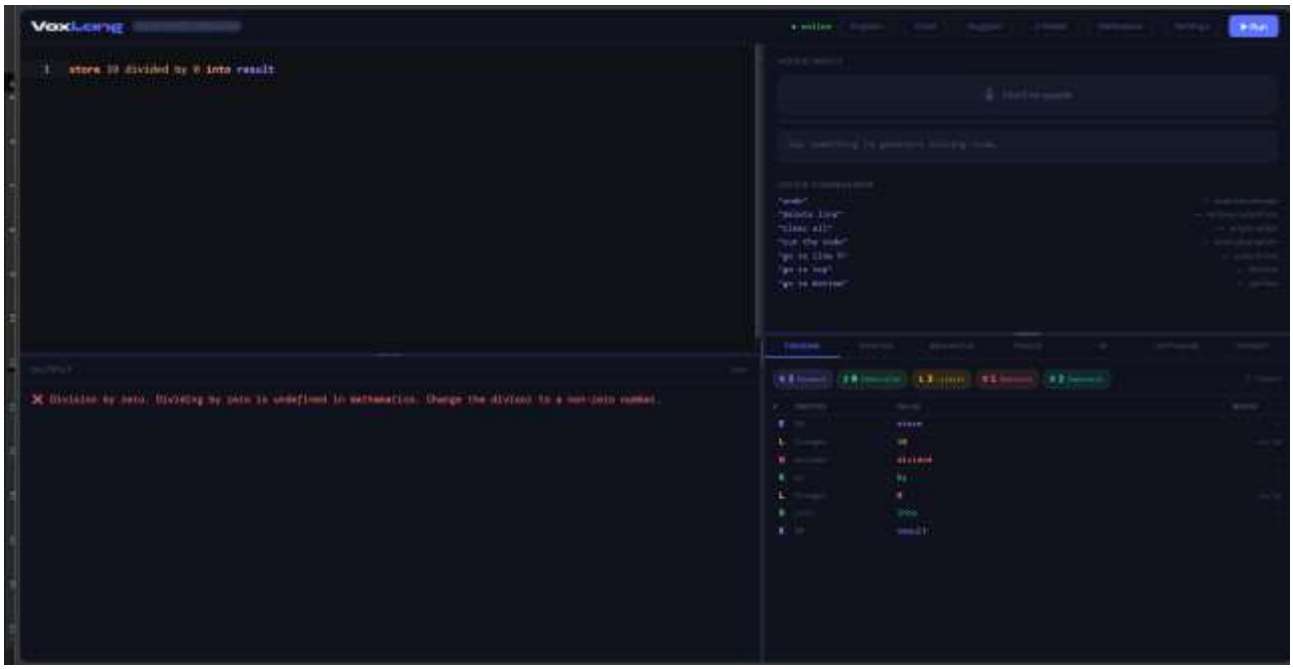
+ 0 folded | 💀 0 dead | ♻ 0 redundant | ✓ 5 kept

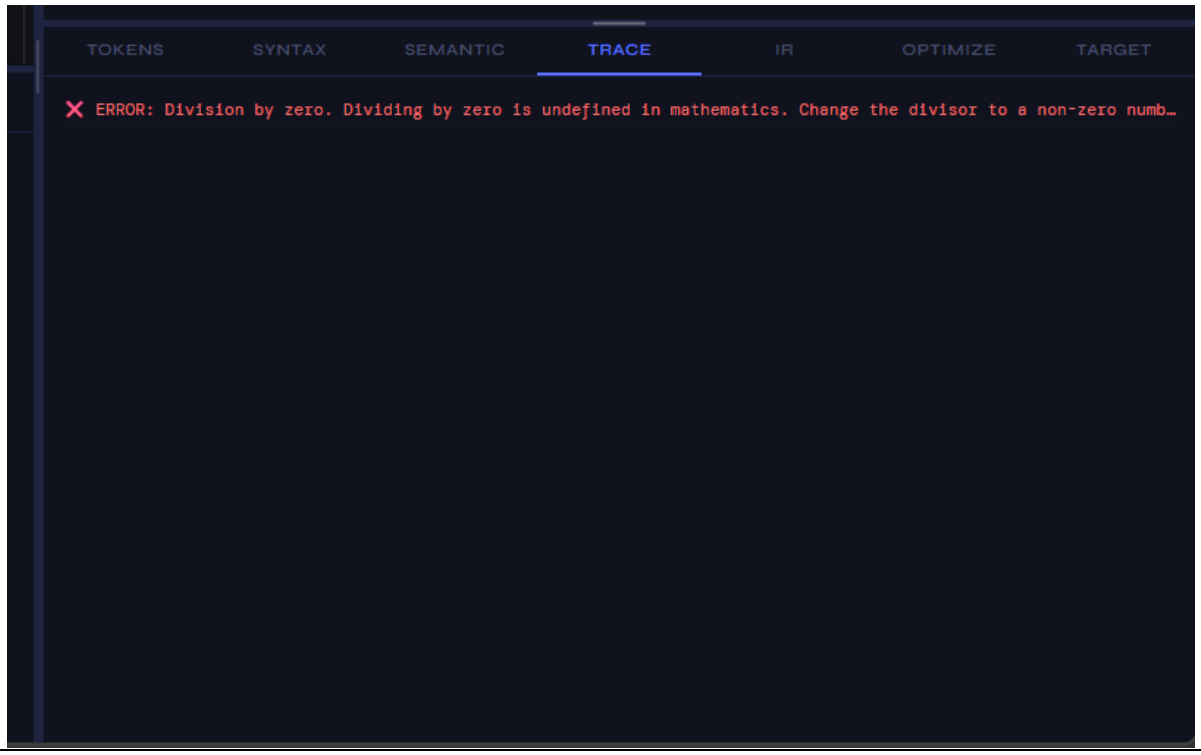
CONSTANT FOLDING ✓ No constant-folding opportunities found.

DEAD STORE ELIMINATION ✓ All stored variables are used — nothing to eliminate.

REDUNDANT LOAD ELIMINATION ✓ No redundant loads detected.

TC2: Syntax Error — missing 'times' **do this** 5 **output** index **done**	Output: 

TC3: Semantic Error — division by zero **store** 10 divided by 0 **into result**	Output: 


TOKENS
SYNTAX
SEMANTIC
TRACE
IR
OPTIMIZE
TARGET
ERROR: Division by zero. Dividing by zero is undefined in mathematics. Change the divisor to a non-zero numb…